

NATIONAL UNIVERSITY OF MODERN LANGUAGES

Faculty of Engineering and Computing
Department of Software Engineering



Operating System

Assignment 03

Submitted To : **Mr Iftikhar Ali**

Submitted By : Ijlal Ahmad
FL-21565

Semester : 4th

Date of Submission : **24 December 2024**

Question # 2 [CLO 3]

1. Definition of Directory Structure and its Purpose in an Operating System

A directory structure in an operating system is a way of organizing and managing files in a computer. It works like a folder system where each file is stored in a specific place, and these places are called directories (or folders). The main purpose of a directory structure is to make it easy for the operating system and users to locate, store, and organize files in a clear and efficient way. Think of it as a map that helps you find your files quickly when you need them.

The directory structure also plays an important role in managing permissions and security, ensuring that files are accessible only to those who have the right to use them. It allows the system to keep everything organized, so files are easy to find, update, or delete.

2. Common Types of Directory Structures

There are three common types of directory structures used by operating systems: single-level, two-level, and tree-structured directories. Let's break each one down:

Single-Level Directory

Description: In a single-level directory, all files are stored in one central directory. There is no subfolder or hierarchy—every file is placed in the same location. This structure is simple but can quickly become messy if there are many files.

Two-Level Directory

Description: A two-level directory structure separates the files into two levels. The first level is the main directory, and inside it, there are subdirectories that contain the files. This allows for a bit more organization, as you can group files into different categories within their respective subdirectories.

Tree-Structured Directory

Description: A tree-structured directory is the most flexible and widely used. It allows for multiple levels of subdirectories, creating a hierarchy where directories can contain other directories, and so on. This structure is useful for large systems, as it allows users to organize files in a way that makes sense for their needs.

Advantages and Disadvantages of Each Directory Structure Type

Single-Level Directory

Advantages:

Simple to implement and manage.

Easy to understand and navigate if there are only a few files.

Disadvantages:

Becomes messy and inefficient as the number of files grows.

Difficult to manage permissions and file access when there are many files.

Hard to locate a specific file quickly when everything is in one place.

Two-Level Directory**Advantages:**

Better organization compared to a single-level structure, with the ability to separate files into categories (like reports, photos, etc.).

Easier to manage file access and permissions for each subdirectory.

Disadvantages:

Still limited compared to more complex structures.

Doesn't handle large numbers of files or deeply nested data well.

Tree-Structured Directory**Advantages:**

Highly organized and flexible, suitable for large and complex systems with many files.

Files can be stored in a hierarchical manner, making it easier to find and manage them.

Supports multiple levels of subdirectories, which makes it scalable as the number of files grows.

Disadvantages:

Can be more complex to manage, especially if the directory tree becomes very deep.

It may be harder for users to understand or navigate if they are not familiar with the structure.

Requires more overhead for the operating system to maintain the hierarchy.

Question # 2 [CLO 3]

1. Creating a Simulated Directory Structure

To create a directory structure for a company using command-line tools, follow these steps:

1. Open the terminal or command prompt.
2. Navigate to the desired location:

```
cd /path/to/directory
```

3. Create the main directory and subdirectories:

```
mkdir Company  
cd Company  
mkdir HR Finance Engineering
```

4. Create files in the respective directories:

```
touch HR/EmployeeData.txt  
touch Finance/Budget2024.xlsx  
mkdir Engineering/Projects  
mkdir Engineering/Projects/ProjectA  
mkdir Engineering/Projects/ProjectB
```

5. Verify the structure using tree:

```
tree /path/to/Company
```

The structure should look like this:

```
Company  
├── HR  
│   └── EmployeeData.txt  
├── Finance  
│   └── Budget2024.xlsx  
└── Engineering  
    ├── Projects  
    │   ├── ProjectA  
    │   └── ProjectB
```

2. Deliverables

Conceptual Questions: Answer questions about directory structures (single-level, two-level, tree-structured).

Screenshots/Logs: Include screenshots or terminal logs showing the creation process.

Comparison: Compare the tree-structured directory to single-level and two-level directories.

Analysis/Recommendations: Explain why a tree-structured directory is best for larger systems, while simpler structures work for smaller ones.

3. Example Document Summary

Conceptual Questions: Explain directory structures and their purpose.

Screenshots/Logs: Show the commands used and directory structure.

Comparison: Discuss the pros and cons of each structure type.

Analysis: Recommend a tree-structured directory for complex systems and a two-level structure for simpler needs.